Taradaciuc Nicolae

Smart Librarian

Project Report

Assignment:

Construiește un chatbot AI care recomandă cărți în funcție de interesele utilizatorului, folosind OpenAI GPT + RAG (ChromaDB), iar apoi completează recomandarea cu un rezumat detaliat obținut printr-un tool separat.

Solution:

1. Project Overview

Smart Librarian is a book recommendation application. It combines a local book catalog, optional Google Books ingestion, OpenAI embeddings, a local Chroma vector store, and OpenAI tool calling to recommend one book and then fetch its exact full summary from a local source.

At a high level, the project does five important things:

a. It builds and maintains a structured local catalog of books in data/book_summaries.json, including optional import and normalization from Google Books.
b. It indexes that catalog in ChromaDB for semantic search.
c. It asks an OpenAI model to choose the best book only from the retrieved candidates.
d. It forces the model to call a local tool, get_summary_by_title(title), so the final full summary comes from the local catalog instead of being invented.
e. It provides a Streamlit interface not only for chatbot interaction, but also for catalog browsing, filtering, direct summary access, and media generation.

The application is exposed through a Streamlit web interface and also includes optional extra features such as:

- local inappropriate-language filtering
- OpenAI moderation
- Romanian and English support
- voice input transcription
- text-to-speech narration
- representative image generation
- Google Books import for catalog expansion
- a full sidebar catalog list with search by title, author, or genre
- direct click on a catalog title to open its exact full summary in the chat

2. Core Functional Idea

The system is not just a chatbot that answers freely. It is a controlled RAG pipeline.

The key design idea is:

- retrieve first
- reason second
- call the local summary tool third
- render a structured answer in the UI

3. End-to-End Runtime Flow

When a user asks a normal question in the Streamlit app, the flow is:

1) The question is typed or transcribed in app_streamlit.py.
2) The UI forwards the question to src.chatbot.chat_once().
3) src.guardrails.py normalizes the text, checks local blocked terms, and validates the input.
4) src.language_support.py detects whether the user is speaking Romanian or English.
5) src.chatbot.py checks whether the message is in scope for a book recommendation app.
6) OpenAI moderation is applied to the input.
7) src.retriever.py embeds the query and asks Chroma for the closest books.
8) src.prompts.py builds a grounded prompt containing only the retrieved candidates.
9) The OpenAI model chooses the best title and must call get_summary_by_title.
10) src.tools.py returns the exact full summary from the local JSON catalog.
11) src.chatbot.py continues the loop, builds the final short explanation, normalizes it to the user's language, and optionally moderates the output.
12) app_streamlit.py renders the final recommendation card, summary, and optional media controls.

4. Catalog Import Flow

The application also supports extending the local book catalog through the sidebar. That flow is:

1. The user opens the "Administrare catalog (Google Books API)" section in the sidebar.
2. The UI loads saved import settings from src/catalog_settings_repository.py.
3. The user selects genres, language restriction, number of books per genre, and page limit.
4. src/catalog_ingestion_service.py receives those settings.
5. It calls scripts/database_loader_script.py to:
   - fetch candidates from Google Books
   - normalize raw metadata into the internal schema
   - infer genres, themes, tone, and audience
   - build short and full summaries
   - merge new books into data/book_summaries.json

6. The in-memory tool cache is refreshed.
7. The Chroma vector store is rebuilt from the validated JSON catalog.

5. Project Features

5.1. Semantic recommendation

The system performs semantic search over book summaries and metadata instead of basic keyword matching. This allows the user to ask questions such as:

- "I want a book about freedom and social control."
- "What do you recommend if I love fantasy stories?"
- "Give me a mystery book."

5.2. Exact full-summary tool calling

The model is not allowed to invent the long summary. It must request it from the local tool.
This is one of the most important design decisions in the project.

5.3. Bilingual interaction

The application supports:

- Romanian
- English

The UI and response payloads are adapted to the detected user language.

5.4. Safety and filtering

The project uses two safety layers:

- local blocked-term filtering
- OpenAI moderation

5.5. Voice interaction

The user can provide an audio question. The project transcribes it and sends the resulting text through the normal chat flow.

5.6. Audio narration

The final recommendation and full summary can be converted into audio using OpenAI TTS.

5.7. Image generation

The user can generate:

- a representative book-cover concept
- a representative scene inspired by the recommendation

5.8. Catalog browsing

The sidebar contains a local catalog browser with:

- title search
- author search
- genre search
- one-click loading of the exact full summary

6. Repository Structure

6.1. Root files

File	Purpose
README.md	Final complete documentation
.env	Local runtime configuration and secrets
.env.example	Safe environment template
.gitignore	Ignore rules for secrets, caches, and local artifacts
requirements.txt	Project dependencies
pytest.ini	Pytest settings
app_streamlit.py	Main web application

6.2. src/ core application layer

File	Purpose
src/config.py	Loads environment variables and shared application settings
src/logger.py	Configures local logging
src/data_loader.py	Loads and validates the JSON catalog
src/embeddings.py	Generates embeddings and OpenAI client access
src/vector_store.py	Creates and rebuilds the Chroma collection
src/retriever.py	Runs semantic search

File	Purpose
src/prompts.py	Builds grounded prompts and system instructions
src/tools.py	Implements exact-title summary lookup
src/guardrails.py	Local validation, filtering, and scope detection
src/language_support.py	Language detection and translation helpers
src/chatbot.py	Main orchestration layer
src/audio_narration.py	TTS narration helpers
src/speech_transcription.py	STT helpers
src/book_image_generation.py	Image prompt building and generation
src/catalog_settings_repository.py	SQLite persistence for import settings
src/catalog_ingestion_service.py	High-level catalog import orchestration

6.3. scripts/

File	Purpose
scripts/database_loader_script.py	Standalone/manual import and normalization script for Google Books
scripts/rebuild_index.py	Manual vector-store rebuild helper

6.4. data/

File	Purpose
data/book_summaries.json	Main local catalog and source of truth
data/blocked_terms_ro.txt	Romanian blocked terms
data/blocked_terms_en.txt	English blocked terms
data/app_state.db	Persisted import settings

6.5. assets/

File	Purpose
assets/avatar_user_green.svg	User avatar in chat
assets/avatar_robot_blue.svg	Assistant avatar in chat

6.6. logs/

File	Purpose
logs/app.log	Local application log

6.7. tests/

File	Purpose
tests/test_chatbot.py	End-to-end chat orchestration coverage
tests/test_guardrails.py	Local filtering and scope detection
tests/test_data_loader.py	Catalog loading and title resolution
tests/test_tools.py	Tool cache and summary lookup
tests/test_catalog_settings_repository.py	Sidebar settings persistence
tests/test_catalog_ingestion_service.py	Import flow refresh behavior
tests/test_language_support.py	Language behavior
tests/test_audio_narration.py	TTS preparation
tests/test_book_image_generation.py	Image generation helpers
tests/test_speech_transcription.py	STT flow

7. Data Model

The internal catalog schema is represented by BookSummary in src/data_loader.py.

Each book contains:

- id
- title
- author
- genres
- themes
- tone
- audience
- language
- short_summary
- full_summary
- content_for_embedding

8. Configuration

8.1. Required environment variables

The application needs an OPENAI_API_KEY.

8.2. Recommended full .env

```
OPENAI_API_KEY=your_real_openai_key
LLM_MODEL=gpt-4.1
EMBEDDING_MODEL=text-embedding-3-small
STT_MODEL=gpt-4o-mini-transcribe
STT_RESPONSE_FORMAT=text
TTS_MODEL=gpt-4o-mini-tts
TTS_VOICE=alloy
TTS_RESPONSE_FORMAT=mp3
MODERATION_MODEL=omni-moderation-latest
IMAGE_MODEL=gpt-image-1-mini
IMAGE_OUTPUT_FORMAT=png
IMAGE_QUALITY=low
IMAGE_STYLE=vivid
IMAGE_COVER_SIZE=1024x1536
IMAGE_SCENE_SIZE=1536x1024
CHROMA_PATH=chroma_db
COLLECTION_NAME=book_summaries
TOP_K=3
ENABLE_INPUT_FILTER=true
MAX_USER_QUERY_CHARS=500
MAX_TOOL_ROUNDS=3
LOG_LEVEL=INFO
```

8.3. Optional Google Books configuration

Optional extra environment variable used only by the Google Books import script:

```
GOOGLE_BOOKS_API_KEY=your_google_books_key
```

That variable is not required for normal chat usage. For this project, the public Google Books endpoint is sufficient for importing books into the local catalog. A Google Books API key is only an optional improvement if you want:

- a more robust import flow
- higher tolerance for repeated requests
- future extension of the Google Books integration
- a more controlled extraction pipeline if you later decide to enrich the dataset further

9. How the Streamlit UI Works

The Streamlit app contains several user-facing areas.

9.1. Main chat area

This is the core interaction area where the user:

- asks for a recommendation
- receives a title, author, genres, explanation, and full summary
- can generate audio or images from the answer

9.2. Starter prompts

When there is no conversation yet, the app shows one-click prompts to help the user start quickly.

9.3. Sidebar catalog administration

The sidebar contains the Google Books import controls:

- genre selection
- number of books per genre
- language restriction
- page count limit

9.4. Sidebar catalog browser

The sidebar also displays the local catalog and lets the user:

- search by title
- search by author
- search by genre
- click a title and load its exact full summary in the chat

9.5. Voice controls

The application provides a floating audio-input control that captures a voice question, transcribes it, and injects the resulting text into the normal recommendation flow.

9.6. Audio controls

Each recommendation can be turned into a playable and downloadable narration.

9.7. Image controls

Each recommendation can also generate:

- a cover-style illustration
- a representative scene

10. Safety and Validation

The application includes both local and remote safety layers.

Local safety:

- input cleaning and normalization
- inappropriate-language filtering with local Romanian and English term lists
- scope detection for book-related questions only
- title and query alias normalization before retrieval
- local enforcement that the tool can only be called for retrieved catalog candidates

Remote safety:

- OpenAI moderation on input
- OpenAI moderation on output

Additional safety-oriented behavior:

- false-positive moderation handling for benign or harmless prompts
- image-generation prompt sanitization and safe-mode retry when a request is blocked
- speech transcription restricted to Romanian and English

This combination makes the system safer than a free-form chatbot because unsafe or irrelevant input is filtered early, and the final long summary always comes from the local catalog instead of being invented by the model.

10. Testing

The tests cover:

- guardrails and scope detection
- moderation behavior
- retrieval + tool-calling orchestration
- summary tool cache reload behavior
- data loading and title normalization
- import settings persistence
- catalog ingestion refresh behavior
- audio generation helpers

- image generation helpers
- speech transcription helpers
- language normalization logic

11. Example Questions

You can test the application with prompts such as:

- Vreau o carte despre libertate si control social.
- Ce-mi recomanzi daca iubesc povestile fantastice?
- Ce este 1984?
- I want a book about friendship and magic.
- Give me a dystopian novel.
- Recommend something for children.

12. Final Conclusion

Smart Librarian is an AI-based recommendation and retrieval solution that combines a validated local book catalog, OpenAI embeddings, a local ChromaDB semantic search layer, and exact-title tool workflow. The local JSON source guarantees trustworthy full summaries, the Chroma layer supports semantic candidate retrieval, and the Streamlit interface offers a practical and accessible experience for end users.

The project presents a RAG + tool-calling architecture and extends it with production-style features such as moderation, guardrails, catalog import, catalog browsing, voice interaction, text-to-speech narration, and image generation.

The final result is a well-structured Smart Librarian application centered on book discovery, grounded summaries, and a complete web-based user workflow